Relatório de Auditoria Técnica

VirtualShop — fuga de dados e preferências entre utilizadores

Data	31 de outubro de 2026
Âmbito	`app/customer_app`, `app/customer_web`, `app/admin_web`, `app/delivery_app` e `backend`
Método	Revisão estática de código + testes Flutter não destrutivos
Restrições	Auditoria read-only; nenhum ficheiro de código foi alterado; testes backend destrutivos não executados
Classificação	Uso interno

Conclusão principal: a API não foi identificada como origem da fuga. Os endpoints privados auditados filtram por `req.user.id` e o middleware valida a sessão (`sid`) contra `user_sessions`. A origem está no estado de sessão dos clientes: caches Riverpod, estado React, preferências globais e refresh de tokens sem geração de sessão.

A. Resumo executivo

- **A origem principal da fuga não está na API.** No código auditado, os endpoints privados filtram por `req.user.id` e o `requireAuth` valida a sessão (`sid`) contra `user_sessions`.
- **A origem está no estado de sessão dos clientes.** No `customer_app`, providers Riverpod com dados privados sobrevivem ao logout/login. No `customer_web`, estado React e `localStorage` também sobrevivem.
- **Tema, idioma e pesquisas recentes são globais**, não por utilizador, e não são repostos no logout.
- Existe ainda um **risco de corrida no refresh de tokens**, capaz de reintroduzir tokens antigos depois do logout ou de contaminar a sessão do utilizador seguinte.
- **Não foram alterados ficheiros.** A auditoria foi estática, complementada por testes Flutter não destrutivos.

B. Âmbito e método

- Projetos analisados: `app/customer_app`, `app/customer_web` (existe no repositório e tem auth próprio), `app/admin_web`, `app/delivery_app` e `backend`.
- Análise de código, rotas, providers, armazenamento local, fluxos de login/logout, refresh de tokens e testes existentes.
- Testes executados: `flutter test` em `customer_app` — **45 testes passaram**.
- Testes backend **não executados**: vários usam `resetDb` e apagam dados reais (`users`, `orders`, `products`, etc.).
- `orders-routes.test.js` aparenta estar quebrado por chamada incorreta a `generateAccessToken`.

C. Achados por severidade

ID	Severidade	Área	Achado
C1	CRÍTICO	`customer_app`	Dados privados de X permanecem em providers após logout/login de Y.

ID	Severidade	Área	Achado
C2	CRÍTICO	`customer_web`	Estado React e dados privados persistem entre contas.
H1	ALTO	Ambos os clientes	Corrida no refresh de tokens pode reintroduzir sessão antiga.
H2	ALTO	`customer_app`	Tema e idioma globais não repostos no logout.
H3	ALTO	`customer_app`	Pesquisas recentes globais em `SharedPreferences`.
H4	ALTO	`customer_app`	Respostas HTTP antigas podem atualizar providers do novo utilizador.
H5	ALTO	`customer_web`	Tokens em `localStorage` e logout sem revogação remota.
M1	MÉDIO	Backend	Role vem do JWT, não do estado atual da BD.
M2	MÉDIO	Backend	Alteração de password não revoga sessões existentes.
M3	MÉDIO	Testes backend	`orders-routes.test.js` chama `generateAccessToken(user)` sem `sessionId`.
M4	MÉDIO	Testes backend	`resetDb` apaga dados; requer base de dados isolada.
M5	MÉDIO	Testes	Falta cobertura de transição X→logout→Y nos clientes.
L1	BAIXO	`customer_app`	Opções de `FlutterSecureStorage` inconsistentes no carrinho local.
L2	BAIXO	Backend	`/auth/refresh` e `/auth/logout` sem rate limiting.

CRÍTICO

C1 — customer_app: dados privados do utilizador anterior permanecem em cache

CRÍTICO

- Providers privados são non-autoDispose e não observam o utilizador autenticado:
  - `profileProvider`, `profileStatsProvider`, `profileAddressesProvider`: `app/customer_app/lib/features/profile/presentation/providers/profile_providers.dart:14-26`
  - `ordersProvider` e `orderDetailProvider`: `order_providers.dart:11-19`
  - `notificationsProvider` e `unreadNotificationCountProvider`: `notification_providers.dart:13-21`
  - `paymentMethodsProvider`: `payment_methods_provider.dart:13-21`
- O logout apenas limpa tokens e coloca `authControllerProvider` a `null`; não invalida estes providers: `auth_providers.dart:84-87`.
- O `ProviderScope` é único e não é recriado por utilizador: `main.dart:15`. O `AuthGate` apenas troca `HomeScreen` / `LoginScreen`: `main.dart:73-80`.
- O `HomeScreen` usa `IndexedStack`, pelo que `ProfileScreen` é construído imediatamente e carrega perfil, estatísticas, endereços e métodos de pagamento: `home_screen.dart:96-104`, `home_screen.dart:130`, `profile_screen.dart:339`, `365`, `476`, `552`.
- Resultado: após login de Y, Y pode ver perfil, endereços, métodos de pagamento, estatísticas, pedidos e notificações de X. O "refresh manual" corrige porque `ProfileScreen._refresh` atualiza esses quatro providers: `profile_screen.dart:31-37`.

C2 — customer_web: estado React e dados privados persistem entre contas

CRÍTICO

- O logout apenas limpa tokens, `user` e muda a página: `app/customer_web/src/main.tsx:28`.
- Não são limpos: `cart`, `addresses`, `favorites`, `noticeItems`, `orderId`: declaração do estado em `main.tsx:11`.
- Notificações de X continuam visíveis para Y porque `noticeItems` não é limpo nem recarregado automaticamente: `main.tsx:30`.
- Favoritos de X persistem no estado local; não são carregados da API no login: `main.tsx:20`, `main.tsx:25`.
- O carrinho só é substituído se o pedido de carrinho de Y tiver sucesso; falhas são ignoradas silenciosamente, deixando o carrinho de X: `main.tsx:12`.
- Endereços de X podem permanecer se o perfil de Y carregar mas o pedido de endereços falhar: `main.tsx:19`.

ALTO

H1 — Corrida no refresh de tokens pode reintroduzir sessão antiga

ALTO

- `DioClient` mantém `_refreshInProgress` sem geração de sessão: `dio_client.dart:112`.
- `_doRefresh` guarda os novos tokens sem verificar se o utilizador mudou ou fez logout: `dio_client.dart:127-147`.
- O logout é “best-effort”: se a chamada ao servidor falhar, limpa apenas o armazenamento local: `auth_repository.dart:86-99`.
- Consequência: um refresh iniciado por X pode gravar tokens de X depois do logout ou depois do login de Y. Se o logout remoto falhar, a sessão de X continua válida e pode haver pedidos com identidade errada. Se o logout remoto tiver sucesso, Y pode ser desautenticado ou ficar com tokens inconsistentes.
- O mesmo padrão existe no `customer_web`: `app/customer_web/src/services/api.ts:9-12`.

H2 — customer_app: tema e idioma são globais e não são repostos

ALTO

- Chaves globais `app_language` e `app_theme_mode`: `app_preferences.dart:37-38`.
- Restauro e gravação sem qualquer âmbito por utilizador: `app_preferences.dart:40-48`, `app_preferences.dart:50-68`.
- O `MaterialApp` observa diretamente este provider: `main.dart:23-36`.
- Resultado: Y herda o tema/idioma de X; o refresh manual não corrige; X também pode herdar alterações feitas por Y.

H3 — customer_app: pesquisas recentes são globais

ALTO

- Chave única `recent_searches` em `SharedPreferences`: `recent_search_repository.dart:4-9`.
- Sem associação ao utilizador e sem limpeza no logout.
- Resultado: histórico de pesquisas de X fica visível para Y no mesmo dispositivo.

H4 — Respostas HTTP antigas podem atualizar providers do novo utilizador

ALTO

- Controladores fazem `await` e escrevem no estado sem verificar geração/utilizador:
  - `ProfileController.refresh`: `profile_providers.dart:32-37`
  - `ProfileStatsController.refresh`: `profile_providers.dart:158-163`
  - `ProfileAddressesController.refresh`: `profile_providers.dart:171-176`
  - `OrdersController.refresh`: `order_providers.dart:26-30`
  - `NotificationsController.refresh`: `notification_providers.dart:39-50`
  - `PaymentMethodsController.refresh`: `payment_methods_provider.dart:23-28`
  - `CartController._reloadRemote`: `cart_provider.dart:297-304`
  - `FavoritesController.reload`: `favorites_provider.dart:65-70`
- Se um pedido de X terminar depois do login de Y, o estado de Y pode receber dados de X.

H5 — customer_web: tokens em localStorage e logout sem revogação remota

ALTO

- Access token e refresh token ficam em `localStorage`: `api.ts:4-5`.
- O logout não chama `/auth/logout`; apenas apaga localmente: `main.tsx:28`.
- Resultado: a sessão continua válida no servidor; o refresh token pode ser reutilizado se for roubado. Também há risco de XSS e de um refresh em curso repor tokens depois do logout.

MÉDIO

M1 — Backend usa o role do JWT, não o role atual da BD

MÉDIO

`req.user = { id: userId, role: payload.role }`: `backend/src/middlewares/auth.middleware.js:50`. Se um administrador for despromovido, o token antigo mantém privilégios até expirar.

M2 — Alteração de password não revoga sessões existentes

MÉDIO

`updatePassword` apenas atualiza `password_hash`: `backend/src/modules/users/users.service.js:165-176`. Sessões antigas continuam válidas até expirarem ou serem revogadas explicitamente.

M3 — Teste de rotas de pedidos está quebrado

MÉDIO

`generateAccessToken` exige `(user, sessionId)`: `backend/src/utils/tokens.js:13-14`. O teste chama `generateAccessToken(user)`: `backend/tests/orders-routes.test.js:61-65`. O teste falharia com `TypeError: sessionId é obrigatório...`.

M4 — Testes backend são destrutivos

MÉDIO

`resetDb` apaga notificações, favoritos, carrinhos, pedidos, pagamentos, endereços, produtos, categorias e utilizadores: `backend/tests/orders-routes.test.js:11-22`. Não devem ser executados contra a base de dados de desenvolvimento/produção.

M5 — Falta cobertura de transição entre utilizadores

MÉDIO

Os 45 testes Flutter passam, mas não cobrem “login X → logout → login Y” para providers privados. Não há teste equivalente no `customer_web`. O teste `auth-sessions.test.js` cobre sessões no backend, mas não valida o comportamento dos clientes.

BAIXO / INFORMATIVO

- **admin_web:** sessão apenas em memória e com geração/guard no refresh (`admin_session.dart:38-44`, `96-112`); não foi identificada fuga entre administradores, mas um refresh da página termina a sessão.
- **delivery_app:** ainda é o template padrão Flutter, sem auth nem dados privados implementados.
- **L1:** `LocalCartRepository` usa `FlutterSecureStorage` com opções diferentes das usadas para tokens (`local_cart_repository.dart:13-16`); a chave é por utilizador, mas há inconsistência de configuração.
- **L2:** `/auth/refresh` e `/auth/logout` não têm rate limiting, ao contrário de login/registo (`auth.routes.js:11-14`).

> **Controlos positivos encontrados no backend:** rotas privadas usam `requireAuth`; serviços de pagamentos, notificações, pedidos, favoritos, carrinho e perfil filtram por `user_id`; `requireAuth` valida `sid`, utilizador, revogação, expiração e conta ativa. Não foi confirmada falha de isolamento nos endpoints auditados, mas falta validação em runtime com base de dados isolada.

D. Causa raiz e reprodução

Causa raiz principal: o cliente trata "logout" como limpeza de tokens, mas não como mudança de âmbito de dados. Caches de providers, preferências, pesquisas e estado React continuam associados ao utilizador anterior.

Reprodução no customer_app

1. Login com X.
2. Abrir Home/Perfil para carregar perfil, estatísticas, endereços e pagamentos.
3. Abrir notificações/pedidos, se aplicável.
4. Logout.
5. Login com Y.
6. Abrir Perfil sem refresh manual: aparecem dados de X.
7. Fazer pull-to-refresh no Perfil: os dados passam a ser de Y.
8. Tema/idioma e pesquisas recentes continuam de X mesmo após refresh.

Reprodução no customer_web

1. Login com X.
2. Carregar notificações/favoritos/endereços/carrinho.
3. Logout.
4. Login com Y.
5. Abrir notificações: lista de X permanece.
6. Favoritos de X permanecem marcados; endereços/carrinho podem permanecer se os pedidos de Y falharem.

E. Plano de correção recomendado

P0 — impedir fuga entre contas

1. **customer_app: criar um "session scope" por utilizador.**
   - Invalidar no login/logout/expiração todos os providers privados: perfil, estatísticas, endereços, pagamentos, pedidos, detalhe de pedido, notificações, contador, carrinho, favoritos e pesquisas.
   - Preferir também `autoDispose` + `ref.watch(authControllerProvider.select((s) => s.valueOrNull?.id))`.
   - Adicionar uma geração de sessão a cada controller e ignorar respostas que cheguem depois de mudança de utilizador.
2. **customer_app: corrigir o refresh de tokens.**
   - Adicionar geração de sessão ao `DioClient`.
   - No logout/login, incrementar a geração, cancelar/ignorar refresh em curso e limpar `_refreshInProgress`.
   - `_doRefresh` só deve guardar tokens se a geração não mudou.
   - O logout deve aguardar/cancelar o refresh antes de limpar o armazenamento.
3. **customer_web: limpar e recarregar todo o estado por utilizador.**
   - Chamar `/auth/logout` antes de limpar tokens.

- Limpar `user`, `cart`, `addresses`, `favorites`, `noticeItems`, `orderId` e mensagens.
- Recarregar favoritos, notificações, carrinho e endereços no login.
- Adicionar geração de sessão ao refresh de `api.ts`.
- Remover tokens de `localStorage`; preferir cookie `HttpOnly` / `Secure` para refresh e access token em memória.

4. **Preferências e pesquisas.**
   - Guardar tema/idioma por utilizador, por exemplo `app_language_$userId` e `app_theme_mode_$userId`, ou repor defaults no logout.
   - Guardar pesquisas recentes por utilizador e limpar no logout, se forem consideradas privadas.

P1 — endurecer backend e testes

5. Validar o `role` atual na BD em operações administrativas ou usar `token_version`/revogação por alteração de role.
6. Revogar sessões em alteração de password.
7. Adicionar rate limiting a `/auth/refresh` e `/auth/logout`.
8. Corrigir `orders-routes.test.js` para `generateAccessToken(user, user.sessionId)`.
9. Executar testes backend apenas em base de dados isolada.
10. Adicionar testes de transição X→logout→Y nos clientes e testes de ownership/forged `sid` no backend.

P2 — qualidade

1. `admin_web`: decidir se a sessão deve persistir; atualmente perde-se ao recarregar.
2. `delivery_app`: implementar ou remover do âmbito de entrega.
3. Uniformizar opções de `FlutterSecureStorage` no carrinho local.

F. Critérios de aceitação

- Após login de Y, nenhum ecrã mostra dados de X antes de refresh manual.
- Tema, idioma e pesquisas recentes pertencem ao utilizador atual ou são repostos no logout.
- Pedidos de X em curso não atualizam estado de Y.
- Refresh iniciado por X não grava tokens depois do logout nem depois do login de Y.
- Logout no `customer_web` revoga a sessão no servidor e limpa todo o estado privado.
- Backend continua a devolver apenas recursos do `req.user.id`; tokens forjados com `sid` de outro utilizador continuam a receber `401`.
- Testes Flutter, React e backend passam, incluindo testes específicos de mudança de conta.
- `npm test` do backend só é executado em base de dados isolada.

Documento gerado a partir da auditoria técnica read-only do repositório VirtualShop. Nenhum ficheiro de código-fonte foi alterado durante a auditoria. Os resultados de runtime limitam-se aos testes Flutter executados; os testes backend destrutivos ficaram deliberadamente por executar.